

Vision-Language Models for Spatial Understanding

Mehregan Nazarmohsenifakori

1 Project Overview

This project studies whether **Vision-Language Models (VLMs)** can act as useful judges for the compositional correctness of images generated by text-to-image diffusion models.

The project focuses on the fact that diffusion models can generate realistic-looking images, but may still fail at **compositional understanding**. Typical failures include:

- incorrect **2D spatial relations**,
- incorrect **3D spatial relations**,
- incorrect **object counts**,
- and failures in more **complex multi-object scenes**.

To study this problem, the project compares:

- two image generators: **SDXL** and **FLUX**,
- an automatic **proxy evaluation pipeline**,
- and multiple **VLM-as-a-Judge** models.

The benchmark categories used in the project are:

- **spatial**
- **3d_spatial**
- **numeracy**
- **complex**

2 Hardware and Runtime Notes

The full experiments in this project are computationally expensive. The main runs were carried out using GPU resources such as:

- **L4 GPU**
- **A100 GPU**

Because of the computational cost of the full pipeline, the repository also includes a compact demo workflow in the final notebook for easier reproduction and inspection.

3 Project Workflow

The project is organized as a sequence of four notebooks.

01_generate_images.ipynb

This notebook is responsible for the **image generation stage**.

Main tasks:

- Load benchmark prompts from the selected categories
- Generate images using:
 - **SDXL**
 - **FLUX**
- Use **3 seeds per prompt** in the main experiments
- Save generated images and create indexing CSV files

Purpose:

- Build the full dataset of generated images used in all later stages
- Make the generation process reproducible and traceable

02_proxy_construction.ipynb

This notebook builds the automatic **proxy evaluation pipeline**.

Main tasks:

- **Step 2:** use **OWL-ViT** for object detection and coarse relation/count reasoning
- **Step 3:** use **SAM3** for segmentation-based refinement of spatial relations
- Build automatic proxy labels for:
 - object presence
 - spatial relations
 - numeracy / object counts

Purpose:

- Provide an automatic reference baseline for evaluating large numbers of generated images

03_vlm_as_judge.ipynb

This notebook evaluates the generated images using **multimodal VLM judges**.

Main tasks:

- Run the following VLM judge models:
 - **Qwen2.5-VL**
 - **Qwen3-VL**
 - **Gemma 3**
- Test multiple prompt variants for judge prompting
- Ask the VLM to produce:
 - a **presence score** (1–5)
 - a **relation / count / composition score** (1–5)

Purpose:

- Study whether VLMs can serve as semantic judges of generated images
- Compare the judge outputs against the automatic proxy pipeline

04_final_demo_and_analysis.ipynb

This notebook is the final integrative notebook of the project.

It has two roles:

1. **Compact end-to-end demo** on a small subset of prompts
2. **Final analysis** of the full experiment outputs

Main tasks:

- Run a small demo pipeline:
 - subset selection
 - image generation
 - Step 2
 - Step 3
 - numeracy proxy
 - one VLM judge
- Load the full saved outputs from the previous notebooks
- Compute final analysis metrics such as:
 - mean proxy scores
 - mean VLM scores
 - normalized VLM scores
 - Pearson / Spearman correlation
 - Cohen's kappa
 - weighted kappa
 - binary accuracy
 - JSON parse rate

Purpose:

- Show a compact reproducible version of the full project pipeline
- Provide the final interpretation of the full experimental results

4 Final Demo Project Folder: final_demo_project

The project includes a dedicated folder named:

`final_demo_project`

This folder is used by the final notebook as a compact and reproducible version of the full pipeline.

Purpose of `final_demo_project`

The full experiments are computationally expensive and produce a large number of generated images and evaluation files. For this reason, the final notebook also uses a smaller demo workflow, whose outputs are stored inside `final_demo_project`.

This folder is useful for:

- demonstrating the full pipeline on a small subset of prompts,
- reproducing the project logic without rerunning the full experiment,
- and inspecting intermediate outputs of the demo pipeline.

What `final_demo_project` contains

The folder typically contains subfolders such as:

- `images/`
 - regenerated demo images
- `tables/`
 - `demo_generated_index.csv`
 - `demo_step2_bbox.csv`
 - `demo_step3_sam3.csv`
 - `demo_numeracy.csv`
 - `demo_demo_vlm_gemma3_prompt_v2.csv`

How it is used in the project

The folder `final_demo_project` is mainly used in `04_final_demo_and_analysis.ipynb`.

In that notebook, it supports the following compact workflow:

1. select a small subset of prompts,
2. regenerate images,
3. run Step 2,
4. run Step 3,
5. run numeracy proxy evaluation,
6. run one VLM judge,
7. and store the intermediate and final demo outputs.

So, while the full experiment results are stored separately, `final_demo_project` acts as the **demo-scale version of the full project pipeline**.

Access Note

The folder `final_demo_project` is **not included directly in the zip submission**, because it contains larger experiment artifacts and demo outputs. To access it, please use the **Google Drive shared link** provided later in this README.

The shared Google Drive folder contains:

- the `final_demo_project` folder,
- generated images,
- saved CSV outputs,
- and other large project artifacts.

5 How the Full Pipeline Connects

The overall project path is:

1. **Generate images** from benchmark prompts
2. **Construct automatic proxy labels** from the generated images
3. **Evaluate the same images with VLM judges**
4. **Compare proxy outputs and VLM outputs**
5. **Analyze** which generators, categories, judge models, and prompts work best

So the project does not rely on only one evaluation view. Instead, it combines:

- generation,
- automatic proxy extraction,
- semantic VLM judging,
- and final comparative analysis.

6 Reproducibility Note

The project includes four notebooks that correspond to the **full workflow**:

1. `01_generate_images.ipynb`
2. `02_proxy_construction.ipynb`
3. `03_vlm_as_judge.ipynb`
4. `04_final_demo_and_analysis.ipynb`

These notebooks were used to produce the complete experimental pipeline and the full saved outputs analyzed in the project.

However, for **rerunning the project in a compact and practical way**, it is sufficient to use only:

`04_final_demo_and_analysis.ipynb`

This last notebook is the recommended entry point for reproduction, because it contains:

- a small end-to-end **demo pipeline**,
- the loading of the saved outputs from the full experiments,
- and the **final analysis** of the project results.

So, while the first three notebooks represent the complete original workflow, the project can be **rerun for demonstration and inspection using only the last notebook**. This is the recommended option for a lightweight and reproducible rerun.

7 Important Note About the Proxy

In the analysis, the proxy pipeline is treated as the **reference ground truth**. However, the proxy is not perfect human annotation. It is an automatic baseline that may still fail because of:

- detection errors,
- segmentation errors,
- counting errors.

Therefore, disagreement between the proxy and a VLM judge should be interpreted carefully. In some cases, the VLM may appear correct by visual inspection even when the proxy gives an incorrect label. Still, for the purpose of this project, the comparison is performed against the proxy outputs as the reference baseline.

8 Google Drive Link

The generated images, saved outputs, and project artifacts can also be accessed through the shared Google Drive folder below:

Google Drive shared link:

<https://drive.google.com/drive/folders/18yYlghZDvjrQn8SyQ9QZkbolxAk78FAa?usp=sharing>

This link can be used to access:

- generated image folders,
- saved CSV outputs,
- demo outputs,
- and additional project artifacts.

9 Final Remark

Taken together, the notebooks provide a complete study of:

- diffusion-model compositional behavior,
- automatic proxy evaluation,
- and VLM-as-a-Judge performance.

The project therefore combines both:

- a reproducible pipeline,
- and a final analysis of the experimental findings.